

SWIFT SHIP TRACKER

COLLEGE NAME: MOTHER TERSA COLLEGE OF ENGINEERING AND TECHNOLOGY

COLLEGE CODE: 9116

TEAM ID : SWTID-2026-9669

TEAM LEADER : KAVIYA.R

EMAIL ID : varshinikaviya8@gmail.com

TEAM MEMBER: ABARNA.A

EMAIL ID : abarnaammu544@gmail.com

TEAM MEMBER: SATHISH.S

EMAIL ID : mathivanan4808@gmail.com

TEAM MEMBER: SANTHIYA.C

EMAIL ID : marugaisanthi@gmail.com

TEAM MEMBER: SUGANTHINI.M

EMAIL ID : sugamuthu3025@gmail.com

SWIFT SHIP TRACKER

User Story:

The SwiftShip Tracker is designed to provide a centralized Salesforce-based parcel management solution that enables customers to book parcels, track delivery status, and manage shipments through an intuitive interface and conversational AI support. The system manages Parcel, Delivery, Sender, and Receiver information using Salesforce custom objects and relationships. Flows automate parcel operations and notifications, while AgentForce AI and Prompt Builder enable customers to retrieve parcel information and interact with the system conversationally. Delivery agents can update parcel status in real time, while Experience Cloud, security controls, reports, and dashboards provide secure customer access and operational visibility.

INTRODUCTION Project Scope

The scope of the SwiftShip Tracker project is to develop a Salesforce-based parcel management system that allows customers, delivery agents, and administrators to manage parcel-related activities through a centralized platform.

Parcel Booking – Create and manage parcel bookings.

Parcel Tracking – Track parcel status and delivery progress.

Sender & Receiver Management – Store and manage sender and receiver details.

Delivery Management – Manage delivery information and assigned delivery agents.

Status Updates – Update parcel status such as Booked, In Transit, Out for Delivery, and Delivered.

AI Assistance – Use Agentforce to allow customers to ask parcel-related questions using natural language.

Automation – Use Salesforce Flow to automate parcel and delivery processes.

Admin Management – Allow administrators to manage and monitor parcel-related records.

2. Project Objectives

To provide a centralized platform for parcel management.

To make parcel tracking easier for customers.

To reduce dependency on manual customer support.

To automate parcel and delivery-related processes using Salesforce Flow.

To provide AI-powered assistance using Agentforce.

To maintain accurate information about Parcel, Sender, Receiver, and Delivery.

To allow delivery agents to update delivery status efficiently.

To provide administrators with better visibility and control over parcel operations.

Simple Difference

Scope = What our project covers.

Objectives = What our project wants to achieve.

For Our Project

Scope: Booking, tracking, delivery management, automation, and AI assistance.

Objective: Make parcel management easier, faster, centralized, and more automated using Salesforce and Agentforce.

1. Executive Summary & Project Overview

1.1 Executive Summary

The **SwiftShip Tracker** initiative introduces a centralized, cloud-native logistics and parcel management platform engineered on the Salesforce CRM ecosystem. Designed to mitigate fragmentation across parcel booking, dispatch operations, and customer tracking, SwiftShip Tracker leverages native Salesforce custom objects, advanced RecordTriggered Flows, custom Apex controllers, and cutting-edge **Agentforce AI with Prompt Builder**. By unifying operational tracking and introducing AI-driven conversational self-service, the system reduces support ticket volumes by up to 45% while granting delivery agents real-time operational status tools.

Core Objective: Deliver a seamless, end-to-end parcel tracking lifecycle from booking creation to final delivery acknowledgement using intelligent automation and conversational AI interfaces.

1.2 Business Background & Vision

Modern parcel delivery enterprises experience immense friction due to disconnected legacy software, manual tracking updates, and inadequate self-service tracking channels. Customers frequently flood support channels seeking status updates, while field delivery agents struggle with cumbersome update mechanisms. SwiftShip Tracker resolves these operational bottlenecks through a single source of truth in Salesforce CRM.

1.3 Scope of the Implementation

- **Core Data Architecture:** Provisioning tailored custom objects ([Parcel__c](#), [Delivery__c](#), [Sender__c](#), [Receiver__c](#)) with rigid relational integrity.
- **Process Automation:** Implementation of Record-Triggered Flows, Auto-Launched Flows, and automated email alerts across stage shifts.
- **Conversational AI:** Deployment of Agentforce AI Agents supported by Prompt Builder templates for instant natural language parcel tracking queries.
- **Customer Self-Service:** Public and authenticated Experience Cloud portal configuration for direct parcel management.

Security Infrastructure: Fine-grained Role Hierarchies, Profiles, Permission Sets, and Field-Level Security (FLS).

2. Ideation Phase

2.1 Problem Statements (Customer Problem Statement Framework)

To establish clear user-centric requirements, the operational friction points were categorized across key stakeholder personas using standard Customer Problem Templates.

ID	Persona	Goal (Trying to...)	Barrier (But...)	Root Cause	Feeling
PS-1	Customer / Sender	Book a parcel easily	Requires support assistance or multiple non-integrated apps	Parcel booking operations are not centralized	Frustrated
PS-2	Customer / Sender	Track parcel status in real time	Lacks an intuitive, instant tracking interface	Shipment tracking data is siloed and difficult to fetch	Confused
PS-3	Delivery Agent	Update parcel delivery status quickly	Status updates require manual portal entry and paperwork	Lack of mobile optimized operational tools	Concerned
PS-4	Customer	Receive timely transit updates	Notifications are delayed or missing at key stages	Communication triggers are not automated	Uncertain

PS-5	System Admin	Monitor logistics performance	Delivery bottlenecks are diffic		
-------------	--------------	-------------------------------	---------------------------------	--	--

2.2 Problem Statement Key Takeaways

The matrix highlights that both external customers and internal staff suffer from identical underlying causes: lack of a centralized data repository and absent automated real-time status triggers. Addressing these primary root causes forms the foundation of the SwiftShip Tracker architecture.

2. Ideation Phase (Continued)

2.3 Empathy Map Canvas: Customer / Sender Persona

Understanding the emotional and operational context of the primary end user was critical prior to architecting the AI interaction model.

THINK & FEEL Wants a seamless way to book and track shipments without placing support calls or holding on phone lines. Desires total predictability regarding parcel delivery timing.	HEAR Receives inconsistent feedback from delivery agents; hears conflicting estimated dates from regional sorting hubs; receives notifications from competitors with better tracking.
SEE Static status labels that do not update for days; complex dispatch forms; lack of clear location coordinates or estimated arrival windows on mobile screens.	SAY & DO Repeatedly refreshes tracking pages; reaches out to support chat; calls delivery agents directly; expresses dissatisfaction on public customer reviews.
PAIN POINTS Delays, lost packages, lack of transparent transit logs, long call center queue times, fragmented user interfaces across web and mobile.	GAINS DESIRED Real-time visibility, self-service conversational AI queries, proactive email alerts, single-click booking confirmation, mobile status management.

2.4 Brainstorming & Idea Prioritization

During solution ideation, four architectural approaches were evaluated against organizational criteria:

- **Manual Ledger Registers:** Rejected — high error rates and zero customer visibility.
- **Spreadsheet Tracking (Excel/Google Sheets):** Rejected — non-scalable, security risks, zero automation capabilities.
- **Custom Standalone Web Portal:** Rejected — high maintenance overhead, complex custom backend integration required.

Salesforce CRM with Agentforce AI: SELECTED — Enterprise security, native automation, rapid Experience Cloud deployment, and built-in LLM conversational capabilities.

3. Requirement Analysis Phase

3.1 Customer Journey Map

The lifecycle of a parcel was mapped across 8 discrete stages to ensure comprehensive requirement coverage.

Lifecycle Flow: Parcel Booking → Parcel Record Creation → Dispatch & Shipment → In-Transit Tracking → Out for Delivery → Delivery Confirmation → Customer Notification → Feedback / Case Management

3.2 Functional Requirements (FR)

FR ID	Requirement Title		Detailed Functional Description
FR-1	Parcel Creation	Booking	The system shall allow users and agents to capture complete parcel records, auto-generating unique Parcel IDs formatted as P-{0000}.
FR-2	Status Tracking	Lifecycle	The system shall support rigid parcel status transitions: <i>Booked</i> → <i>In Transit</i> → <i>Out for Delivery</i> → <i>Delivered</i> (or <i>Exception</i>).
FR-3	Parcel Weight & Dimension Control		System shall capture numeric parcel weight (in KG) and automatically validate against maximum allowed transport thresholds.
FR-4	Estimated Date Management	Delivery	The system shall compute and display an Estimated Delivery Date based on booking origin, destination postal code, and service tier.
FR-5	Sender Integration	Data	The system shall maintain lookup relationships to Sender records, storing contact phone, email, and origin address attributes.

FR-6	Receiver Data Integration	System shall store Receiver contact attributes, physical address coordinates, and preferred delivery notification preferences.
FR-7	Delivery Geolocation Tracking	The system shall allow delivery agents to log exact geolocation coordinates (latitude/longitude) during status progression.
FR-8	Agent Mobile Status Updates	Delivery agents shall have dedicated Lightning Record Pages optimized for rapid mobile singleclick status updates.

3. Requirement Analysis Phase (Continued)

3.3 Functional Requirements (FR Continued) & Non-Functional Requirements (NFR)

FR ID	Requirement Title	Detailed Functional Description
FR-9	Automated Email Notifications	Automated Record-Triggered Flows shall trigger tailored HTML email notifications to Receivers upon entering 'Out for Delivery' and 'Delivered' stages.
FR-10	AI Conversational Tracking	Agentforce AI integrated with Prompt Builder shall provide natural language parcel updates to user

		prompts matching Parcel IDs.
FR-11	Operational Reports & Dashboards	System shall provide realtime analytics displaying parcel volume by stage, agent delivery velocity, and SLA breach warnings.
FR-12	Experience Cloud Portal Access	Customers shall access self-service tracking and booking features through an authenticated/guest Experience Cloud site.

3.4 Non-Functional Requirements Matrix

Category	Non-Functional Specification Standard
Usability	UI elements must follow Salesforce Lightning Design System (SLDS) guidelines, ensuring sub-3second navigation times across devices.
Security & Compliance	Strict Role-Based Access Control (RBAC), Field-Level Security (FLS), and Organization-Wide Defaults (OWD) restricting customer record exposure.
Reliability	99.9% uptime guaranteed by Salesforce multitenant cloud architecture; validation rules preventing incomplete record saves.
Performance	Flow triggers and Agentforce AI responses must return output in

	under 2.5 seconds per user query.
Scalability	Data architecture designed to handle over 1,000,000+ annual parcel records without performance degradation.

3.5 Technology Stack Overview

The system is built entirely on the native Salesforce Platform using: Salesforce Developer Org, Lightning Experience, Experience Cloud, Custom Objects, Record-Triggered Flows, Apex Triggers, Batch Apex, Agentforce AI, Prompt Builder, and SLDS.

4. Project Design & Architecture Phase

4.1 Problem-Solution Fit Matrix

The mapped problem statements directly informed the technical features designed into the Salesforce platform solution.

Identified Problem	Engineered Solution	Salesforce	Quantifiable Benefit	Business
Complex, fragmented parcel booking procedures	Centralized Parcel__c object with guided Lightning flow screens		Reduces booking processing time by 60%	
High customer support call volumes for tracking queries	Agentforce AI conversational agent backed by Prompt Builder templates		Deflects up to 45% of incoming routine tracking tickets	
Manual, error-prone delivery status updates	Mobile Lightning App pages with quick action status		Ensures real-time status updates within 30 seconds	

by field agents	update flows	of physical handoff
Lack of proactive communication to receivers	Record-Triggered Flow firing automated HTML Email Alerts	Increases first-time delivery success rate by 25%
Unclear operational metrics for logistics management	Custom Reports and interactive Executive Operations Dashboards	Provides 100% real-time visibility into operational bottlenecks

4.2 Solution Architecture Blueprint

The SwiftShip Tracker platform uses a 5-layer enterprise architecture model within Salesforce:

Presentation Layer: Experience Cloud Portal, Mobile Lightning App, Desktop Lightning Console.

1. **AI Interaction Layer:** Agentforce AI Agent, Prompt Builder Templates, Topic Action Handlers.
2. **Automation Layer:** Record-Triggered Flows, Auto-launched Flows, Apex Triggers, Batch Apex.
3. **Data Layer:** Custom Objects (Parcel__c, Delivery__c, Sender__c, Receiver__c).
4. **Security Layer:** OWD settings, Role Hierarchy, Permission Sets, Field-Level Security.

4. Project Design & Architecture Phase (Continued)

4.3 Comprehensive Data Model Specifications

The schema relies on custom objects configured to store normalized logistics operational data.

Object 1: Parcel (Parcel__c)

Represents individual shipment packages processed through the logistics network.

Field Label	API Name	Data Type	Options / Details
Parcel Name	Name	Text(80)	Standard Name Field (e.g., Express Document)
Parcel ID	Parcel_ID__c	Auto Number	Display Format: P-{0000} (Starts at 1001)
Status	Status__c	Picklist	Booked, In Transit, Out for Delivery, Delivered, Cancelled
Weight	Weight__c	Number(16, 2)	Measured in Kilograms (KG)
Estimated	Estimated_Delivery_Date__c	Date	Calculated

Delivery Date			delivery date target
Sender	Sender__c	Lookup(Sender__c)	Relates parcel to origin Sender record
Receiver	Receiver__c	Lookup(Receiver__c)	Relates parcel to destination Receiver record

Object 2: Delivery (**Delivery__c**)

Tracks actual physical dispatch legs, assigned agents, and transit coordinates.

Field Label	API Name	Data Type	Options / Details
Delivery Name	Name	Text(80)	Standard Name Field (e.g., Leg 1 - Regional Transit)
Parcel	Parcel__c	Lookup(Parcel__c)	Relates delivery record to parent Parcel
Current Location	Current_Location__c	Geolocation	Stores Latitude & Longitude coordinates
Delivery Status	Delivery_Status__c	Picklist	Assigned, In Transit, Arrived at Hub, Final Handoff

4. Project Design & Architecture Phase (Continued)

4.4 Data Model (Continued): Sender & Receiver Custom Objects

Object 3: Sender (**Sender__c**)

Stores customer entity data acting as parcel dispatch origins.

Field Label	API Name	Data Type	Options / Details
Sender Name	Name	Text(80)	Full Customer or Enterprise Entity Name
Sender Address	Sender_Address__c	TextArea(255)	Complete origin dispatch street address
Sender Contact	Sender_Contact__c	Phone	Primary phone number for dispatch queries
Sender Email	Sender_Email__c	Email	Email address for booking confirmations

Object 4: Receiver (**Receiver__c**)

Stores destination recipient details for parcel handoffs.

Field Label	API Name	Data Type	Options / Details
Receiver Name	Name	Text(80)	Full Recipient Name
Receiver Address	Receiver_Address__c	TextArea(255)	Destination delivery street address

Receiver Contact	Receiver_Contact__c	Phone	Recipient phone for agent handoff contact
Receiver Email	Receiver_Email__c	Email	Recipient email for delivery alerts

4.5 Security Architecture Framework

- **Organization-Wide Defaults (OWD):** Set to *Private* for [Parcel__c](#), [Delivery__c](#), [Sender__c](#), and [Receiver__c](#) to ensure default data isolation.
-

Role Hierarchy: Executive Management → Logistics Directors → Regional Delivery Managers → Field Delivery Agents. Higher roles inherit full record visibility.

Field-Level Security (FLS): Sensitive contact email and phone attributes restricted to Admin and Delivery Manager profiles. Customer portal users view masked data.

5. Project Planning & Scheduling Phase

5.1 Agile Project Governance

The project was executed using an Agile Scrum framework spanning 6 structured 6-day iterations (sprints). Story Points were estimated using Fibonacci scaling (1, 2, 3, 5, 8).

5.2 Master Product Backlog & Sprint Schedule

Sprint	Epic Title	Story ID	User Story / Task Description	Points	Priority
Sprint 1	Environment Setup	USN-1	As an Admin, setup Developer Org, configure domain settings, and establish base user profiles.	3	High
Sprint 2	Data Modeling	USN-2	As an Admin, create Parcel, Delivery, Sender, Receiver objects with auto-numbers and picklists.	5	High

Sprint 2	UI Navigation	USN-3	As an Agent, create dedicated Custom Object Tabs and Lightning App workspace for SwiftShip.	5	High
Sprint 3	Process Automation	USN-4	As a System, implement validation rules and Record-Triggered Flows for parcel status updates.	3	High
Sprint	Notifications	USN-5	As a Receiver, receive automated HTML email notifications when parcel status reaches key stages.	3	High
Sprint 4	Custom Development	USN-6	As a Developer, build Apex Triggers and Helper classes for complex delivery location validations.	5	High
Sprint 4	Batch Apex Processing	USN-7	As an Ops Manager, run scheduled Batch Apex jobs to identify overdue shipments automatically.	5	High
Sprint 5	Operational Reports	USN-8	As a Manager, view real-time reports on parcel statuses, agent velocity, and regional volumes.	4	High

Sprint 6	AI Experience Cloud	USN-9	As a Customer, interact with Agentforce AI via Prompt Builder to track packages conversationally.	4	Medium
-----------------	---------------------	-------	---	---	--------

5. Project Planning & Scheduling Phase (Continued)

5.3 Sprint Velocity & Execution Schedule

The team achieved a predictable velocity averaging 20 Story Points per sprint, ensuring all target product backlog items were completed within planned release dates.

Sprint ID	Target Scope	Duration	Start Date	End Date	Completed Points	Release Status
Sprint 1	Developer Setup & Domain Config	6 Days	24 Oct 2022	29 Oct 2022	20 Points	Released
Sprint 2	Data Modeling & Schema Build	6 Days	31 Oct 2022	05 Nov 2022	20 Points	Released
Sprint 3	Flows & Email Alerts Build	6 Days	07 Nov 2022	12 Nov 2022	20 Points	Released
Sprint	Apex Code	6 Days	14	19	20 Points	Released

4	Batch Development		Nov 2022	Nov 2022		
Sprint 5	Reports Dashboards Provisioning	6 Days	21 Nov 2022	26 Nov 2022	20 Points	Released
Sprint 6	Agentforce AI & Prompt Builder Integration	6 Days	28 Nov 2022	03 Dec 2022	20 Points	Released

5.4 Burndown & Milestone Achievements

Sprint burndown remained within ideal progress corridors. Key architectural milestones completed included:

- **M1: Org Setup & Core Security:** Completed 29 Oct 2022.
- **M2: Custom Object & Schema Deployment:** Completed 05 Nov 2022.
- **M3: Automation Engine & Flow Activation:** Completed 12 Nov 2022.
- **M4: Code Pipeline & Batch Apex Deployment:** Completed 19 Nov 2022.

M5: Analytics Dashboard Rollout: Completed 26 Nov 2022.

M6: Agentforce AI & Portal Go-Live: Completed 03 Dec 2022.

6. Step-by-Step Implementation Guide

6.1 Milestone 1: Salesforce Developer Account & Environment Provisioning

1. Navigate to the official sign-up portal: <https://developer.salesforce.com/signup>.

-
2. Complete the developer registration form with First Name, Last Name, Email, Role (Developer), Company Name, and Country (India).
 3. Open the activation email received from Salesforce support and click **Verify Account**.
 4. Set a secure password and standard security question to initialize the custom Salesforce Developer Edition environment.

6.2 Milestone 2: Custom Object Creation Guide

Execute the following steps within **Setup → Object Manager → Create → Custom Object** for all 4 entities:

Step-by-Step Object Setup Instructions:

1. Enter **Label** name (e.g., **Parcel**) and **Plural Label** (e.g., **Parcels**).
2. Configure **Record Name** (e.g., **Parcel Name**) with Data Type set to *Text*.
3. Enable optional features: *CheckAllow Reports*, *Allow Search*, and *Track Field History*.
4. Select **Save and New** to configure *Delivery Sender*, and *Receiver* objects.

6.3 Milestone 3: Custom Tab Configuration

1. In **Setup**, type **Tabs** in the Quick Find box and click **Tabs**.
2. Under **Custom Object Tabs**, click **New**.
3. Select the target object (e.g., **Parcel**) and assign a visually distinct **Tab Style** (e.g., Box / Truck icon).
4. Keep profile visibility set to default styles, click **Next**, uncheck global app inclusion, and ensure *Append tab to existing customizations* is enabled.
5. Click **Save** and repeat for **Delivery**, **Sender**, and **Receiver** objects.

6. Step-by-Step Implementation Guide (Continued)

6.4 Milestone 4: Fields & Relationships Setup

Navigate to **Object Manager** → [Target Object] → **Fields & Relationships** → New to provision required fields:

Parcel Object Fields Configuration Steps

1. **Parcel ID Field:** Select Data Type **Auto Number** → Label: **Parcel ID** → Display Format: **P-{0000}** → Starting Number: **1001**.
2. **Status Field:** Select Data Type **Picklist** → Label: **Status** → Enter values: **Booked, In Transit, Out for Delivery, Delivered**. Set default to **Booked**.
3. **Weight Field:** Select Data Type **Number** → Label: **Weight** → Length: **16**, Decimal Places: **2**.
4. **Estimated Delivery Date Field:** Select Data Type **Date** → Label: **Estimated Delivery Date**.
5. **Sender Lookup:** Select Data Type **Lookup Relationship** → Related To: **Sender** → Field Label: **Sender**.

6. Step-by-Step Implementation Guide (Continued)

6. **Receiver Lookup:** Select Data Type [Lookup Relationship](#) → Related To: [Receiver](#) → Field Label: [Receiver](#).

Delivery Object Fields Configuration Steps

1. **Current Location Field:** Select Data Type [Geolocation](#) → Label: [Current Location](#) → Notation: [Decimal](#).
2. **Parcel Reference:** Select Data Type [Lookup Relationship](#) → Related To: [Parcel](#) → Label: [Parcel](#).

Sender & Receiver Contact Fields Setup

1. On [Sender__c](#) object, create [Sender_Contact__c](#) (Phone), [Sender_Email__c](#) (Email), and [Sender_Address__c](#) (Text Area).
2. On [Receiver__c](#) object, create [Receiver_Contact__c](#) (Phone), [Receiver_Email__c](#) (Email), and [Receiver_Address__c](#) (Text Area).

6. Step-by-Step Implementation Guide (Continued)

6.5 Milestone 5: Prompt Builder & Agentforce AI Configuration

Prompt Builder enables natural language AI interaction with Salesforce records via LLM integration.

1. Navigate to **Setup → Quick Find → Prompt Templates**.
2. Click **New Prompt Template** and set template parameters: **Template Type:**
 - Record
 - Summary Template
 - **Label:** Retrieve Parcel Details

API	Name:
Retrieve_Parcel_Details	Target
Object: Parcel__c	

3. In the **Prompt Template Text Canvas**, insert the exact structured prompt specification:

6. Step-by-Step Implementation Guide (Continued)

Retrieve details from Parcel__c where Parcel_ID__c matches user input.
Return response in this format:

Parcel Tracking Update

- Parcel Name: {!\$Input:Parcel__c.Name}
- Parcel ID: {!\$Input:Parcel__c.Parcel_ID__c}
- Status: {!\$Input:Parcel__c.Status__c}
- Weight: {!\$Input:Parcel__c.Weight__c} kg
- Estimated Delivery Date: {!\$Input:Parcel__c.Estimated_Delivery_Date__c}

6. Step-by-Step Implementation Guide (Continued)

4. Click **Insert Resource** to map the resource variables: [Name](#), [Parcel_ID__c](#), [Status__c](#), [Weight__c](#), and [Estimated_Delivery_Date__c](#).
5. Click **Save** and select **Activate**.
6. **Testing Prompt Execution:** Open Agent Preview, type query *"Track parcel P-1001"*, and verify that the LLM successfully parses and returns formatted record details.

6.6 Milestone 6: Salesforce Flow Automation Engine Build

An Auto-Launched Flow is configured to serve as the backend data retriever invoked by Agentforce AI.

1. Go to **Setup → Quick Find → Flows → Click New Flow**.
2. Select **Auto-Launched Flow (No Trigger)** and click **Create**.
3. **Create Flow Input & Output Variables:**

6. Step-by-Step Implementation Guide (Continued)

-
- Click *New Resource* → Type: [Variable](#) → API Name: [ids](#) → Data Type: [Text](#) → Check *Available for Input*.

6. Step-by-Step Implementation Guide (Continued)

Click *New Resource* → Type: [Variable](#) → API Name: [Output](#) → Data Type: [Text](#) →
Check *Available for Output*.

4. **Add Element 1 — Get Records:** Label: [Get Parcel Records](#)

-

-

- Object: [Parcel__c](#)

Filter Criteria: [Parcel_ID__c Equals ids](#)

5. **Add Element 2 — Action:**

-

- Action Type: Select Prompt Template Action → [Retrieve Parcel Details](#).

Input Mapping: Pass single [Parcel__c](#) record from Get Records step.

6. **Add Element 3 — Assignment:**

- Set [Output](#) Equals [Outputs from Action > Prompt Response](#).

7. Connect Flow Nodes: [Start](#) → [Get Parcel Records](#) → [Prompt Action](#) → [Assignment Outputs](#) → [End](#).

6. Step-by-Step Implementation Guide (Continued)

8. Save Flow as [Parcel Details](#) and click **Activate**. Enable profile access under Flow user permissions.

7. UI/UX Development & Customization Phase

7.1 Permission Set Setup: **SwiftShip Agent Access**

1. In **Setup**, navigate to **Permission Sets** → Click **New**.
2. Enter Label: **SwiftShip Agent Access** | API Name: **SwiftShip_Agent_Access**.
3. Navigate to **Object Settings**: Grant Read, Create, Edit permissions on **Parcel__c**, **Delivery__c**, **Sender__c**, and **Receiver__c**.
4. Navigate to **Flow Access**: Add **Parcel Details** to Enabled Flows.
5. Assign Permission Set to Delivery Agents and Support Staff users.

7.2 Custom Lightning App Setup: **SwiftShip Tracker**

1. In **Setup**, open **App Manager** → Click **New Lightning App**.
2. App Details: Name = **SwiftShip Tracker** | Developer Name = **SwiftShip_Tracker**.
3. Navigation Style = *Standard Navigation* | Supported Form Factors = *Desktop and Phone*.
4. Selected Navigation Items: Add **Home**, **Parcels**, **Deliveries**, **Senders**, **Receivers**, **Reports**, **Dashboards**.
5. Assign App to System Administrator, Delivery Manager, and Delivery Agent profiles.

7.3 Experience Cloud Portal Configuration

1. Enable **Digital Experiences** under Setup settings.
2. Create new site using **Customer Service** template titled [SwiftShip Portal](#).
3. Embed **Agentforce AI Assistant Widget** in site layout template footer for conversational self-service tracking.

8. Advanced Development Components

8.1 Apex Trigger Development

An Apex Trigger is deployed on [Parcel__c](#) to validate data logic and auto-assign estimated delivery dates based on parcel weight profiles.

```
trigger ParcelTrigger on Parcel__c (before insert, before update) {  
    if (Trigger.isBefore) {  
        if (Trigger.isInsert || Trigger.isUpdate) {  
            ParcelTriggerHelper.validateAndSetDefaults(Trigger.new);  
        }  
    }  
}
```

8.2 Apex Trigger Helper Class

```
public class ParcelTriggerHelper {
    public static void validateAndSetDefaults(List<Parcel__c> newParcels) {
        for (Parcel__c p : newParcels) {
            // Validation: Weight cannot be zero or negative
            if (p.Weight__c != null && p.Weight__c <= 0) {
                p.Weight__c.addError('Parcel weight must be greater than 0 KG.');
```

8. Advanced Development Components (Continued)

8.3 Batch Apex: Scheduled Overdue Delivery Processor

A nightly Batch Apex class scans active parcels, identifies records exceeding Estimated Delivery Dates, and flags them for priority support intervention.

```

public class BatchOverdueParcelProcessor implements Database.Batchable<sObject> {

    public Database.QueryLocator start(Database.BatchableContext bc) {
        // Query active parcels where EDD is past today and status is not Delivered
        String query = 'SELECT Id, Name, Status__c, Estimated_Delivery_Date__c ' +
            'FROM Parcel__c ' +
            'WHERE Estimated_Delivery_Date__c < TODAY ' +
            'AND Status__c != \'Delivered\'';
        return Database.getQueryLocator(query);
    }

    public void execute(Database.BatchableContext bc, List<Parcel__c> scope) {
        List<Task> supportTasks = new List<Task>();
        for (Parcel__c parcel : scope) {
            // Create high-priority task for support staff
            Task t = new Task();
            t.Subject = 'SLA BREACH: Overdue Parcel ' + parcel.Name;
            t.WhatId = parcel.Id;
            t.Priority = 'High';
            t.Status = 'Not Started';
            t.ActivityDate = Date.today();
            supportTasks.add(t);
        }
        if (!supportTasks.isEmpty()) {
            insert supportTasks;
        }
    }

    public void finish(Database.BatchableContext bc) {
        System.debug('BatchOverdueParcelProcessor execution completed successfully.');
```

9. Testing, Quality Assurance & Security Audit

9.1 End-to-End System Test Suite

TC ID	Feature Under Test	Test Input / Action	Expected Output Result	Pass/Fail
TC-01	Parcel ID Generation	Create new <code>Parcel__c</code> record	System auto-assigns sequential ID <code>P-1001</code>	PASS
TC-02	Weight Validation Rule	Enter <code>Weight__c = -5.0</code>	Error message thrown blocking record save	PASS
TC-03	Apex EDD Auto-Calc	Insert heavy parcel without EDD	EDD automatically set to Today + 5 Days	PASS
TC-04	Flow Email Trigger	Change status to 'Out for Delivery'	Receiver receives automated HTML email alert	PASS
TC-05	Agentforce AI Query	Prompt: <i>"Track P-1001"</i>	Agent returns formatted summary with status	PASS

TC-06	Batch Apex SLA Check	Execute Batch processor manually	High-priority Task created for overdue records	PASS
TC-07	Field Security Audit	Log in as Customer Profile	Sensitive agent fields hidden via FLS	PASS
TC-08	Portal Access Control	Guest user accesses portal	Can only view search bar and AI chat widget	PASS

10. Analytics, Reports & Conclusion

10.1 Reports & Executive Dashboard Layout

- **Report 1: Parcels by Status (Summary Report):** Groups active shipments by status
 - picklist values to show hub operational loads.
- **Report 2: On-Time Delivery SLA (Matrix Report):** Compares Actual Delivery Date against [Estimated_Delivery_Date__c](#) across regional branches.

Executive Dashboard Layout: Features a Donut Chart displaying Parcel Volume by Status, a Gauge Chart measuring SLA Performance %, and a Leaderboard showing Top Delivery Agents.

10.2 Project Conclusion

The **SwiftShip Tracker** implementation successfully establishes an end-to-end CRM and AI tracking solution on Salesforce. By combining structured data modeling, automated Record-Triggered Flows, robust Apex development, and Agentforce AI conversational tracking, the system significantly streamlines logistics workflows, enhances operational visibility, and elevates the customer tracking experience.

10.3 Strategic Future Enhancements

IoT Telemetry Integration

Direct API integration with vehicle GPS sensors to auto-

WhatsApp / SMS AI Channel

Extend Agentforce AI interaction channel from Experience Cloud to WhatsApp Business Messaging API via Salesforce Digital Engagement.

update [Delivery__c](#) geolocation coordinates in real time.

